Terraform: IaC, state, drift và review plan

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Terraform và ý nghĩa của state

Infrastructure as Code (IaC) là mô tả hạ tầng bằng file có thể review và lưu lịch sử. Terraform đọc cấu hình mong muốn, đối chiếu với state và hệ thống thật, rồi tạo một plan các thay đổi dự kiến. **plan** không phải lời bảo đảm tuyệt đối về mọi kết quả thực thi; môi trường có thể thay đổi giữa lúc lập plan và apply. Người thực hiện phải đọc plan, nhất là mục thay thế hoặc xóa.

Một resource là đối tượng Terraform quản lý; provider nói chuyện với API của nền tảng. Variable giúp thay đổi đầu vào; output công bố thông tin cần dùng; module đóng gói một phần cấu hình. Bắt đầu với cấu hình nhỏ trước khi tạo nhiều lớp module. Module phức tạp quá sớm khiến người mới khó nhìn thấy tài nguyên thật được tạo ra.

State lưu ánh xạ giữa resource trong cấu hình và đối tượng thực. State có thể chứa dữ liệu nhạy cảm, nên cần kiểm soát quyền và nơi lưu. Khi nhiều người cùng sửa, locking giúp tránh ghi đè đồng thời. Xóa state file không tương đương xóa hạ tầng; tự ý sửa state có thể khiến Terraform hiểu sai môi trường. Drift là khác biệt do thay đổi ngoài quy trình; phát hiện drift trước khi apply để tránh ghi đè ngoài ý muốn.

Lab Terraform local trước cloud

Điều kiện: đã cài Terraform và có một provider an toàn cho lab. Chọn hướng dẫn chính thức "Get Started" phù hợp môi trường. Đọc file cấu hình, chạy **terraform init**, **terraform fmt**, **terraform validate**, **terraform plan**. Trước apply, ghi lại số tài nguyên tạo/sửa/xóa và những giá trị chưa biết. Chỉ apply với tài nguyên thử nghiệm mà bạn hiểu cách dọn. Sau đó sửa một thuộc tính, quan sát plan; cuối cùng dùng **terraform destroy** đúng workspace/lab và kiểm tra tài nguyên thực đã biến mất. Đừng commit state hay biến chứa secret.

Tình huống điều tra

Plan báo "replace" một database. Dừng lại và tìm thuộc tính gây thay thế. Hỏi dữ liệu hiện tại ở đâu, có backup/restore đã thử chưa, tác động downtime là gì và có cách migration khác không. Một câu trả lời tốt trong phỏng vấn không phải "chạy apply" mà là giải thích cách đánh giá tác động và chọn thời điểm/chiến lược an toàn.

Lab Terraform không tốn phí cloud

Thư mục **lesson/labs/terraform-local** có một **main.tf** dùng resource tích hợp **terraform_data**. Resource này lưu một giá trị trong Terraform state và không tạo tài nguyên cloud. Nhờ đó bạn có thể học vòng lặp init -> plan -> apply -> thay đổi -> destroy trước khi sử dụng tài khoản có tính phí. Đây là lab về mô hình hoạt động của Terraform, chưa phải bài thiết kế hạ tầng thực.

Mở terminal ở thư mục này. **terraform version** cho biết công cụ có sẵn hay chưa. **terraform init** chuẩn bị thư mục làm việc. **terraform fmt -check** kiểm tra định dạng; **terraform validate** kiểm tra cú pháp và một số quy tắc cấu hình. **terraform plan** cho xem dự kiến một **terraform_data.label** sẽ được tạo. Đọc ba dòng quan trọng: tên resource, giá trị input và tổng số create/change/destroy. Chỉ sau khi hiểu plan,

chạy **terraform apply** trong lab này và xác nhận. Dùng **terraform output resource_label** để xem kết quả mong đợi **notes-lab**.

Sửa default của variable từ **lab** thành **demo**, chạy plan lại. Trước khi xem output, dự đoán Terraform sẽ tạo mới, cập nhật hay thay resource. Đọc plan thực tế và giải thích tại sao. Quay lại **lab** nếu muốn, rồi chạy **terraform destroy** trong đúng thư mục lab; plan phải cho biết đối tượng state nào sẽ bị xóa. Vì lab dùng **terraform_data**, thao tác destroy chỉ bỏ state tương ứng, không xóa máy/cloud. Đừng suy rộng điều này sang các resource cloud khác.

State file local được tạo sau apply. Dùng **terraform state list** để xem địa chỉ resource; có thể dùng **terraform show** để đọc state theo cách công cụ hỗ trợ. Không chỉnh state bằng editor. Nếu thấy file state trong **git status**, thêm quy tắc ignore phù hợp trước khi commit. State của project thật có thể chứa dữ liệu nhạy cảm và phải được lưu/khóa theo chính sách nhóm.

Nếu lab lỗi

Nếu **terraform** không được nhận diện, công cụ chưa cài hoặc chưa nằm trong PATH; dùng hướng dẫn cài chính thức cho hệ điều hành, rồi mở terminal mới. Nếu **init** lỗi tải provider, kiểm tra kết nối mạng và proxy; lab tích hợp có thể không cần provider ngoài nhưng Terraform vẫn cần chuẩn bị môi trường. Nếu **plan** báo lock, xem có tiến trình Terraform khác đang chạy không; không tự xóa lock của team. Với lab cá nhân, đọc thông báo để hiểu trường hợp cụ thể.

State, drift và thay đổi ngoài quy trình

Terraform state là trí nhớ của Terraform về các đối tượng mà nó quản lý. Nếu ai đó sửa tài nguyên qua console, cấu hình trong Git chưa thay nhưng hệ thống thật đã khác. Lần plan sau có thể cho thấy Terraform muốn đưa hệ thống về trạng thái cấu hình; đó là một dấu hiệu drift. Tuy nhiên, không phải mọi thay đổi hiển thị trong plan đều do người sửa tay: provider có giá trị tính toán, API có mặc định và phiên bản provider có thể thay đổi. Đọc diff cụ thể trước khi kết luận.

Trong lab **terraform_data**, đổi giá trị **environment** rồi chạy plan để thấy khác biệt có chủ ý. Quay lại file cũ nhưng chưa apply và quan sát plan đổi. Bài này mô phỏng vòng lặp cấu hình/state. Với cloud thật, còn một nguồn trạng thái nữa là tài nguyên trong API cloud. Đó là lý do cần quản lý quyền thay đổi thủ công và review plan thường xuyên.

Không đưa **terraform.tfstate** lên Git. Nếu hai người cùng apply vào cùng môi trường bằng hai bản state độc lập, mỗi người có thể hiểu sai quyền sở hữu tài nguyên. Remote state với locking phù hợp giúp nhóm dùng chung trạng thái, nhưng quyền đọc state cũng cần hạn chế vì secret có thể xuất hiện trong đó. Đánh dấu output **sensitive** giúp giảm lộ trên CLI nhưng không thay thế bảo vệ state.

Review plan như một thay đổi sản xuất

Một plan cần được đọc theo bốn tầng. Tầng 1: đúng account, region, workspace và commit không? Tầng 2: số resource tạo/sửa/xóa/thay thế là bao nhiêu? Tầng 3: thuộc tính nào gây tác động, đặc biệt địa chỉ public, IAM permission, database, storage và backup? Tầng 4: nếu apply thất bại giữa chừng hoặc kết quả khác dự kiến, cách phát hiện và phục hồi là gì? Bài lab có thể bỏ qua một số bước vì không tạo cloud, nhưng thói quen đọc plan cần bắt đầu từ đây.

Giả sử plan thêm một security rule cho phép **0.0.0.0/0** tới cổng database. Tên resource có thể trông hợp lệ, nhưng nội dung rule nguy hiểm. Người review cần hiểu đường traffic dự kiến và yêu cầu sửa về nguồn hẹp hơn. Giả sử plan xóa một storage bucket chứa backup; dừng và kiểm tra dữ liệu, retention, owner, mục đích. Không đo chất lượng IaC bằng việc **terraform apply** kết thúc thành công; đo bằng kết quả hệ thống và rủi ro đã kiểm soát.

Bicep, ARM và Terraform trong Azure

ARM là nền tảng quản lý/deploy tài nguyên Azure; ARM template dùng JSON. Bicep là ngôn ngữ khai báo gọn hơn cho Azure, biên dịch thành ARM template. Terraform dùng state riêng và provider để quản lý Azure cùng các nền tảng khác. Không nên gọi Bicep là "Terraform của Microsoft" rồi bỏ qua khác biệt state, lifecycle và cách preview. Khi làm việc trong tổ chức, lựa chọn còn chịu ảnh hưởng governance, kỹ năng đội ngũ và hệ thống hiện có.

Bicep có what-if để xem thay đổi dự kiến; Terraform có plan. Cả hai cần review và phân quyền. Không đặt bí mật trong file tham số được commit. Nếu một tài nguyên được quản lý bởi cả hai công cụ hoặc sửa tay thường xuyên, drift và quyền sở hữu trở nên mơ hồ. Quy định rõ tài nguyên nào do pipeline nào quản lý.

Bài tập so sánh

Lấy một resource group và một storage account làm ví dụ. Viết bảng: nơi khai báo, cách preview, trạng thái lưu ở đâu, ai apply, cách xử lý rollback và drift. Không cần thuộc cú pháp hết; mục tiêu là hiểu quy trình thay đổi. Sau đó đọc một mẫu Bicep chính thức và một mẫu Terraform Azure, chỉ ra resource, parameter/variable và output.

Mental model: configuration, state, real infrastructure

Terraform có ba hình ảnh cần so sánh: mã cấu hình mô tả mong muốn, state lưu mapping resource/thuộc tính mà Terraform biết, và hạ tầng thực tế qua provider API. **plan** tính chênh lệch giữa ba hình ảnh; **apply** thực hiện và cập nhật state. Nếu ai sửa resource bằng portal, lần plan sau có thể phát hiện drift. Nếu state mất hoặc bị sửa tay, Terraform có thể tưởng resource chưa tồn tại và định tạo lại. State có thể chứa dữ liệu nhạy cảm; bảo vệ bằng quyền, mã hóa, versioning và backend/locking phù hợp.

Đọc plan theo từng resource và loại hành động: create, update in-place, replace, destroy. Với **replace**, hỏi vì sao thuộc tính buộc thay mới, downtime và dữ liệu sẽ đi đâu. Một plan "2 to add, 0 to change" chưa đủ; resource mới có thể là public IP hoặc rule mở rộng. Đặc biệt tránh tự động apply một plan chưa review vào production. Plan dựa trên trạng thái tại thời điểm tạo; nếu hạ tầng đổi sau đó, cần lập plan lại hoặc dùng quy trình saved plan có kiểm soát.

Lab providerless và bằng chứng idempotency

Trong **lesson/labs/terraform-local**, chạy **terraform init**, **terraform fmt -check**, **terraform validate**, **terraform plan**. Đọc **main.tf**: resource nào có địa chỉ nào, input/output là gì. Apply trên lab local sau khi xem plan; chạy plan lần hai, mong đợi không có thay đổi. Sửa một giá trị trong cấu hình, plan lại và dự đoán action. Sau đó **terraform destroy** chỉ sau khi xác nhận đang ở đúng thư mục/state lab. Ghi output plan trước/sau và điều gì khiến bạn tin lần hai idempotent. Lab providerless dạy workflow/state, không chứng minh bạn đã cấp cloud resource thật.

Backend, locking và cộng tác

Local state đơn giản cho một người lab, nhưng đội cần backend chia sẻ có quyền/phiên bản và locking hoặc cơ chế tuần tự tương ứng để hai người không apply đồng thời lên cùng state. Tách state theo vòng đời/ranh giới sở hữu phù hợp; một state khổng lồ khiến plan chậm và blast radius lớn, quá nhiều state nhỏ làm dependency khó quản lý. Module giúp tái dùng cấu hình, nhưng module trừu tượng quá mức có thể giấu resource thật khỏi người review. Trong PR, hiển thị plan, ai review, ai apply, state nào bị tác động và cách revert.

Nếu deploy fail giữa chừng, đừng tự xóa state entry để "chạy lại". Đầu tiên xem resource thật đã tạo gì, state ghi gì, lỗi provider nào, rồi chọn import/move/taint/recreate phù hợp với tài liệu phiên bản. Một thao tác state sai có thể gây destroy ngoài ý muốn ở lần apply sau. Quyền cloud của pipeline Terraform nên giới hạn theo môi trường; không dùng tài khoản Owner toàn tổ chức cho một lab.

Terraform với Ansible và Helm

Terraform tạo network/VM/managed cluster; Ansible cấu hình OS/package trên máy; Helm triển khai manifest ứng dụng trong Kubernetes. Đây là một ranh giới thường gặp, không phải luật cứng. Quan trọng là một tài nguyên/field có một owner rõ. Nếu Terraform và Helm cùng quản lý một namespace hoặc Helm release, hoặc người vận hành sửa tay thường xuyên, drift/chuỗi trách nhiệm khó hiểu. Lập bảng "ai tạo, ai sửa, ai xóa, ai backup" cho network, cluster, namespace, app, DB trước khi tự động hóa.

Câu hỏi phỏng vấn có lời giải

"Terraform plan báo sẽ thay DB instance, bạn làm gì?" Trả lời: dừng apply; đọc thuộc tính buộc replace; đánh giá dữ liệu/downtime, backup và restore; xem có cách thay đổi in-place hoặc migration; kiểm tra đúng workspace/backend và drift; đưa plan cho owner review; chỉ thực hiện sau khi có cửa sổ và rollback/runbook. Không trả lời đơn giản "apply vì IaC luôn đúng".

State là sổ cái ánh xạ, không phải bản sao mã nguồn

Terraform có ba thứ: **configuration** mô tả mong muốn, **state** ánh xạ resource address tới ID/thông tin bên provider, và **hạ tầng thật** ở API cloud/on-prem. Khi bạn đổi HCL, Terraform dùng state và đọc API để tính plan. Nếu xóa file resource khỏi code, plan có thể đề nghị destroy object đang được state quản lý. Nếu một người sửa trực tiếp ở portal, plan có thể phát hiện drift; nhưng các thuộc tính do provider không đọc được hoặc bỏ qua có thể không xuất hiện theo cách bạn kỳ vọng. State chứa dữ liệu nhạy cảm tùy resource, vì vậy không commit vào Git và cần quyền truy cập, encryption/backup phù hợp.

Resource address như **module.network.azurerm_subnet.app** là danh tính logic trong state. Đổi tên block không có nghĩa muốn xóa subnet thật; nếu không chuyển state/moved block đúng cách, plan có thể đề nghị destroy/create. Import đưa object có sẵn vào quản lý nhưng không tự viết cấu hình đầy đủ hay bảo đảm thay đổi kế tiếp an toàn. Trước khi áp dụng cho resource production, review plan và ràng buộc dữ liệu/phụ thuộc.

Đọc plan theo tác động, không theo số dòng

+ là create, **~** update, **-** destroy, **-/+** replacement. Một plan “1 to add, 1 to destroy” có thể là thay VM tạm ít rủi ro hoặc thay database chứa dữ liệu; con số không cho biết mức độ. Với mỗi replacement hỏi: vì sao ForceNew, dữ liệu/địa chỉ IP có đổi không, downtime bao lâu, dependency nào theo sau, backup đã được restore thử chưa? **terraform plan -out=...** lưu kế hoạch cụ thể để apply lại, nhưng saved plan cũng có thể chứa thông tin nhạy cảm và chịu ảnh hưởng thay đổi ngoài luồng; bảo vệ như artifact nhạy cảm. Không tự động **apply -auto-approve** ở production khi chưa có review chính sách phù hợp.

Lab review không tốn cloud

Điều kiện: Terraform CLI đã cài, dùng thư mục **lesson/labs/terraform-local** riêng; lab mẫu hiện có cấu hình local. Đọc **main.tf** và dự đoán resource/output. Chạy **terraform init**, **terraform fmt -check**, **terraform validate**, **terraform plan**. **Kết quả dự kiến:** init tải provider/module nếu mạng cho phép; validate kiểm tra cấu trúc; plan hiển thị thay đổi dự kiến, chưa tạo gì. Ghi version CLI/provider và thông báo thực. Sau khi hiểu plan, **terraform apply chỉ trong lab local** và xác nhận object/output. Chạy plan lần nữa: nếu cấu hình và thực tế ổn định, dự kiến không có thay đổi. Đó là một kiểm tra idempotency, không phải chứng minh mọi provider luôn không drift.

Thay một thuộc tính vô hại trong lab, chạy plan, đọc trước/sau, rồi apply. Sau đó chỉnh object bên ngoài Terraform nếu provider local cho phép để tạo drift, hoặc mô phỏng bằng cách thay file được quản lý; chạy plan và giải thích hướng Terraform muốn reconcile. Dọn bằng **terraform destroy chỉ khi cwd và state đúng thư mục lab**; đọc plan destroy trước khi chấp thuận. Đừng chạy trên backend/cloud chưa kiểm tra. Giữ bản ghi plan đã làm sạch secret và giải thích tại sao từng thao tác là create/update/delete.

Cộng tác: backend, lock và pipeline

Local state phù hợp học một mình nhưng dễ mất và không hỗ trợ nhiều người cùng apply. Remote backend cần quyền tối thiểu, versioning/backup, encryption và locking nếu backend hỗ trợ. Lock ngăn hai writer đồng thời làm hỏng state. Khi thấy lock bị giữ, xác minh ai đang chạy và run nào còn sống; không dùng **-lock=false** hoặc force-unlock để "chạy cho xong". Tài liệu HashiCorp chỉ cho force-unlock khi xử lý **lock của chính mình** sau khi unlock tự động thất bại; xem hướng dẫn state locking.

Pipeline Terraform có bước plan trên PR, reviewer xem impact và policy, apply từ revision/plan được duyệt với identity phù hợp. Việc plan ở một thời điểm và apply nhiều ngày sau có thể gặp thay đổi thực tế; cần refresh/kiểm tra lại theo quy trình. Tách state theo phạm vi thay đổi/failure domain và quyền sở hữu, nhưng đừng chia cực nhỏ khiến dependency qua state khó quản lý. Output dùng để nối module/hệ thống, tránh đưa secret qua output công khai.

Terraform, Bicep, Ansible và Helm

Terraform/Bicep/ARM thường tạo cloud resources; Ansible cấu hình host hoặc chạy nhiệm vụ; Helm đóng gói manifest Kubernetes. Ranh giới có thể chồng lấp nhưng một resource nên có owner rõ. Nếu Terraform tạo Helm release rồi Argo CD cũng quản lý cùng release, hai controller tranh desired state. Chọn một owner cho từng lớp. Với Azure enterprise, Bicep đi sát Azure Resource Manager; Terraform hữu ích khi đội dùng nhiều provider hoặc module chung. So theo yêu cầu review, state, quyền và kỹ năng đội chứ không có công cụ thắng tuyệt đối.

Ca có lời giải: plan đề nghị xóa database sau khi bạn đổi tên module. Đừng apply. So địa chỉ cũ/mới, state binding và **moved**/state move phù hợp; backup và kiểm tra dependency. **Ca khác:** plan báo no changes nhưng app lỗi; Terraform chỉ mô tả hạ tầng được quản lý, không chứng minh DNS, dữ liệu hoặc app đúng. Nguồn: Terraform state, Terraform plan.

Plan review có lời giải: thay subnet kéo theo database

PR đổi CIDR subnet và **terraform plan** hiện một subnet **-/+**, hai NIC update và một database private endpoint replacement. Dù tác giả nói "chỉ đổi mạng", tác động có thể cắt kết nối DB. Reviewer cần xác định force-replacement reason, IP/DNS, route, downtime, data plane, dependency và rollback; xem môi trường nào dùng subnet này và có tài nguyên ngoài state hay không. Nếu DB endpoint được recreate, DNS TTL/cache và firewall rule có thể làm downtime lâu hơn apply. Không approve theo số dòng HCL nhỏ.

Phương án: thiết kế subnet mới song song nếu provider/kiến trúc cho phép, chuyển workload theo batch, cập nhật DNS/route, smoke test, rồi xóa cũ sau thời gian quan sát. IaC giúp khai báo cả trạng thái trung gian; đôi khi cần nhiều PR/apply. Một **apply** lớn "atomic" trên giấy thực ra là nhiều API call, có thể fail giữa chừng; state và hạ tầng thực cần được kiểm tra sau lỗi.

Sensitive output và state

variable "password" { sensitive = true } có thể che hiển thị trong CLI theo một số ngữ cảnh, nhưng state vẫn có thể chứa secret. Vì vậy bảo vệ backend, quyền đọc state, plan artifact, log CI và backup; không gửi **terraform.tfstate** cho người khác để "debug nhanh". Nếu secret rotation làm state/plan đổi, cân nhắc tham chiếu secret manager phù hợp thay vì lưu value rõ trong resource khi provider cho phép.

Câu hỏi: vì sao **terraform apply** báo thành công mà app chết? IaC xác nhận API hạ tầng theo khai báo, không chứng minh request path/DB/schema. **Câu hỏi:** có nên force-unlock khi pipeline khác đang chạy? Không; hai writer có thể làm hỏng state. Nguồn: Terraform state.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://developer.hashicorp.com/terraform/language/state
- https://developer.hashicorp.com/terraform/cli/commands/plan

Phụ lục: mã lab để đọc offline

lesson/labs/terraform-local/main.tf

```
terraform {
  required_version = ">= 1.4.0"
}

variable "environment" {
  type    = string
  default = "lab"
}

resource "terraform_data" "label" {
  input = "notes-${var.environment}"
}

output "resource_label" {
  value = terraform_data.label.output
}
```